

Rapport individuel — Itération 2

Alexis Briend (abd7852) — Projet Long SHDL

11 mai 2026

Contexte et objectif de l'itération

Le projet long SHDL vise un simulateur de circuits logiques avec interface JavaFX. L'équipe compte neuf personnes réparties sur six axes (éditeur, parser, automates, simulateur, IHM, intégration). Ma responsabilité demeure la **structure générale** et l'**interprétation du SHDL** (ticket SCRUM-23). L'itération 1 avait posé un parser à descente récursive vers un AST. L'objectif de l'itération 2 était double : (i) remplacer ce parser par une version *table-driven* stricte, conforme à l'enseignement de compilation, et (ii) brancher concrètement le parser au simulateur amont via une couche de conversion. Ces deux objectifs ont été atteints et fondés sur 82 commits.

Activités réalisées

- **Refonte complète du parser en LL(1) table-driven** (`parser/ll1/tabledriven/`) : construction de la table d'analyse $M[NT, T]$ à partir des ensembles FIRST et FOLLOW (`TableBuilder`, lookup en $O(1)$), driver à pile générique sans *switch* par non-terminal (`CstParser`). L'ancien parseur récursif est conservé en branche `feature/parser-ll1-old` mais marqué obsolète.
- **Concrete Syntax Tree (CST)** en remplacement de l'AST : types `sealed CstNode = CstLeaf | CstInternal` (records immuables), méthodes de navigation `first(Symbol) / allOf(Symbol) / has(Symbol) / startOffset() / endOffset()`. Trivia (whitespace, commentaires) filtrés avant parsing, offsets préservés. Outil de débogage `CstDumper` avec sortie indentée.
- **Conversion CST $\rightarrow$ simulateur.Module** (`parser/conversion/`, spécification `docs/specs/2026-05-06-cst-c`) : couche *intégration* qui produit l'IR du simulateur de Mati à partir du CST. Périmètre S1 : **combinatoire scalaire stricte**. Les hors-scope (vecteurs, concaténation, mémoire, appel de module, littéraux en RHS, doublons) sont rejetés par une `ConversionException` typée (sept raisons énumérées, offset et type de nœud portés).
- **Bug fix amont** : `simulateur.Erwan.Erwan.OR()` retournait `Op.AND`. Corrigé dans le commit 37cc8d8 avec test sentinelle anti-régression.
- **Documentation pipeline** : `README.md` de la branche réécrit pour exposer en trois lignes le pipeline destiné au bundle JavaFX :

```
CstNode cst    = CstParser.parse(shdlSource);
Module module = Conversion.convert(cst);
FileSimulateur sim = new FileSimulateur(module.Plan);
```

- **Spécifications et plans versionnés** : `docs/specs/2026-04-30-ll1-table-driven-cst-design.md`, `docs/specs/2026-05-06-cst-conversion-design.md`, `docs/plans/` associés.

Résultats

- **193 tests JUnit verts** (107 au sprint 1, soit **+86 tests**), répartis sur 39 classes. Couverture : parser CST (offsets, ε -productions, erreurs avec position vérifiée, multi-instance), conversion (cas nominaux scalaires ET/OU/NOT, rejets exhaustifs hors S1 avec offsets), bout-en-bout (`xor` table de vérité 4 lignes via `FileSimulateur`, fixtures `.shdl` sur disque conformes à la spéc.).
- **14 fichiers Java** ajoutés sous `parser/conversion/` et `parser/ll1/tabledriven/` (hors tests).
- Une **intégration réussie** avec le code amont d'Erwan (`parser/lexer/` via `CstParser.parse`) et de Mati (`simulateur.Module`, `FileSimulateur`). Le piège `TERM_REST` relevé au sprint 1 disparaît : la grammaire LL(1) est gelée par `GrammarFreezeTest` et vérifiée sans conflit FIRST/FIRST ou FIRST/FOLLOW.
- **Écarts assumés vis-à-vis de la grammaire amont** (`shdl_grammar_LL1_v2.txt`) documentés dans `parser/ll1/tabledriven/README.md` : typo `Factor` parens, NTs morts retirés, casse uniformisée,

opérateur mémoire `:=` corrigé en `:=`.

Difficultés et réussites

La principale difficulté a été d'absorber les *instabilités du code amont* : la branche `interpretation` d'Erwan a livré des stubs successifs, et le bug `Erwan.OR()` cité ci-dessus n'a été détecté que par un test e2e sur une table de vérité `XOR`. La discipline TDD — écriture du test négatif avant le passage — a été payée plusieurs fois ce sprint.

La réussite principale est la **séparation stricte des rôles** entre parser, CST, conversion et IR simulateur. Le CST sert d'interface contractuelle : la grammaire peut évoluer sans casser la conversion tant que les méthodes de navigation tiennent. La conversion peut grandir vers le séquentiel ou les vecteurs sans toucher au parser. Cette découpe a permis de paralléliser quatre vagues d'agents de vérification automatique (revue de spec, revue d'implémentation, smoke tests, tests d'intégration) sans pollution du contexte.

Limitation honnête : le scope S1 est *volontairement* pauvre. Le séquentiel (bascules, `:=`), les vecteurs, les appels de module sont acceptés par le parser mais refusés par la conversion. L'extension est mécanique mais nécessitera un IR enrichi côté simulateur (pas à ma main).

Manuel utilisateur

Pré-requis : JDK 25, JUnit 4.13.2 et Hamcrest 1.3 (fournis dans `lib/`).

Compiler et lancer la batterie de tests :

```
./build.sh test      # compile parser + conversion + simulateur + tests
./run.sh             # 193 tests JUnit
```

Convertir un programme SHDL en circuit prêt à simuler :

```
import parser.ll1.tabledriven.CstParser;
import parser.conversion.Conversion;
import simulateur.Module;
import simulateur.FileSimulateur;

CstNode cst      = CstParser.parse(shdlSource);      // ParsingException possible
Module module    = Conversion.convert(cst);          // ConversionException possible
FileSimulateur sim = new FileSimulateur(module.Plan); // implements Simulateur
```

Les `Exception` exposées portent l'`offset()` dans la source, le contexte syntaxique (`contextPath()` côté parser, `nodeKind()` côté conversion) et un code d'erreur structuré, exploitables directement par l'IHM pour le surlignage.

Le code source est publié sur la branche `feature/parser-ll1-table-driven`. La spécification détaillée de la conversion se trouve dans `docs/specs/2026-05-06-cst-conversion-design.md`.